6.2.2 Adding a user profile

When designing a user profile object, you want to think about what kind of data needs to be associated with the user account. Some data in your system will be strictly associated with your user and nothing else. For example, the user's date of birth isn't covered in Django's `User Model` object and might only be associated with that account. Alternatively, if your system associates all sorts of information about birthdays together and represents both the user's birthday as well as their friends who don't necessarily have accounts, you might have a birthday object. In this case, the user profile would contain a key to the user's birthday object instead.

Consider who the RiffMates users are and the kinds of things they do on the site. Some questions to answer are, can a person

- Have more than one musician profile?
- Control more than one venue?
- Be both a musician and a venue operator?

I'm sure there are many more, but those three are enough to think about what a user profile object should contain. If you convert a `Musician` object into a user profile, you can't answer "yes" to any of the preceding questions. It is best to create a separate `Model` object simply for user data and use relationships between it and the `Musician` and `Venue` objects. The following listing shows just such an object; it has a one-to-one relationship with the `User` object and many-to-many relationships with `Musician` and `Venue` objects.

Listing 6.1 The `UserProfile` class

```
# RiffMates/bands/models.py
from django.db import models
from django.contrib.auth.models import User
...
class UserProfile(models.Model):
    user = models.OneToOneField(User,
        on_delete=models.CASCADE)
    musician_profiles = models.ManyToManyField(Musician,
        blank=True)
    venues_controlled = models.ManyToManyField(Venue, blank=True)
```

Import Django's User model object.

Create a one-to-one relationship between the UserProfile and the User.

Store relationships between this user account and any Musician or Venue objects. Set blank=True, so these can be empty fields in the Django Admin.

As the comment in the listing implies, this code gets added to the RiffMates/bands/models.py file. To make user management easier, the Django Admin understands the relationship between a user profile object and the user account. It allows you to define a form to be used within the user account management screens to deal with the associated data. To use this feature you need to define a *stacked inline form* and tell the Django Admin to use it.